

SmartFINN Installation Manual

Version: final software-quality submission

Target platform: Docker Engine with Docker Compose v2

1. Purpose

This manual gives a tester the steps required to install, configure, start, verify and stop SmartFINN. The local installation runs the Next.js frontend and Flask backend in Docker. Supabase supplies authentication, PostgreSQL and object storage; Azure Document Intelligence supplies live OCR.

2. Prerequisites

Install the following before starting:

- Docker Engine or Docker Desktop;
- Docker Compose v2 (`docker compose`);
- a test Supabase project with its URL, browser-safe publishable key and server-only secret key;
- an Azure Document Intelligence test resource for live OCR; and
- approximately 4 GB free disk space.

Git and access to the private course repository are optional when using the supplied Moodle ZIP archive.

The tester must never place a service-role, secret, Azure or Mercury credential in frontend variables or commit a populated `.env` file.

3. Obtain the source

3.1 Moodle ZIP archive (recommended for assessors)

Extract the supplied `SmartFINN_Software_Quality_<revision>.zip` archive, then enter its top-level directory:

```
unzip SmartFINN_Software_Quality_<revision>.zip
cd SmartFINN
```

The archive is generated directly from the final `main` revision and does not contain `.git`, populated environment files, dependency caches or credentials.

3.2 Private repository (team members)

Team members with repository access may instead clone the final `main` branch:

```
git clone git@github.com:unsw-cse-comp99-3900/capstone-project-26t2-9900-w19b-bread.git
cd capstone-project-26t2-9900-w19b-bread
git checkout main
```

For a Git checkout, confirm the expected revision before testing:

```
git status --short --branch
git log -1 --oneline
```

The working tree should be clean and the checked-out branch should be `main`. For a Moodle archive, the revision is recorded in `docs/ACCEPTANCE_TEST_REPORT.md` and in the ZIP filename.

4. Obtain test-environment access

SmartFINN intentionally does not package authentication or cloud-service secrets. The nominated assessor account and test-environment values must be delivered through the private Moodle submission comments or another tutor-approved private channel.

The team should provide:

- an active test staff email and temporary password;
- the test Supabase URL, publishable key and server-only secret key for local Docker testing;
- Azure Document Intelligence test credentials if live OCR is included in the agreed test; and
- the deployed test URL, currently `https://aimeshlab.su.se/bread`, when the tutor has approved testing the hosted instance.

Do not place any of these secret values in the ZIP, Git history, screenshots or acceptance report. Mercury live-write credentials are not required for the standard assessment path; keep Mercury in dry-run mode unless the tutor provides or approves a dedicated Mercury test tenant.

5. Configure Supabase

Apply the SQL files below to a new test Supabase project in dependency order:

- 1 `supabase/migrations/create_smartfinn_schema.sql`
- 2 `supabase/migrations/add_internal_staff_rols.sql`
- 3 `supabase/migrations/add_document_storage_and_intake_rpc.sql`
- 4 `supabase/migrations/add_ocr_response_storage.sql`
- 5 `supabase/migrations/add_rules_engine_phase1.sql`
- 6 `supabase/migrations/add_mercury_sync_workflow.sql`

Do not run `supabase db push` against the existing hosted project until its migration history has been baselined. For a new isolated test project, execute the files in the Supabase SQL Editor and stop immediately if a statement fails.

Create a test staff member:

- 1 Create a user in Supabase Authentication.
- 2 Open `supabase/admin/bootstrap_first_admin.sql`.
- 3 Set `admin_email` to that user's email.
- 4 Execute the script in the Supabase SQL Editor.
- 5 Confirm that it returns one active administrator.

Run the read-only verification scripts under `supabase/verification/` before using the application.

6. Configure environment variables

Create the root environment file:

```
cp deploy/production.env.example .env
```

Set the following values:

Variable	Required	Purpose
SUPABASE_URL	Yes	Test Supabase project URL

Variable	Required	Purpose
SUPABASE_PUBLISHABLE_KEY	Yes	Browser-safe Supabase key
SUPABASE_SECRET_KEY	Yes	Server-only backend key
AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT	For live OCR	Azure resource endpoint
AZURE_DOCUMENT_INTELLIGENCE_KEY	For live OCR	Server-only Azure key
AZURE_DOCUMENT_INTELLIGENCE_MODEL	For live OCR	Normally prebuilt - layout
AZURE_DOCUMENT_INTELLIGENCE_API_VERSION	For live OCR	2024-11-30 for the tested configuration
AZURE_DOCUMENT_INTELLIGENCE_REGION	For live OCR	Resource region, such as australiaeast

Mercury is optional. Keep these safe defaults unless a dedicated test tenant has been provided:

```
MERCURY_ENABLED=false
MERCURY_ALLOW_WRITES=false
MERCURY_DRY_RUN=true
```

Restrict access to the environment file on Unix-like systems:

```
chmod 600 .env
```

7. Build and start

Validate the resolved Compose configuration:

```
docker compose config --quiet
```

Build and start both services:

```
docker compose up --build -d
```

Allow up to two minutes for the first build and health checks. Inspect status:

```
docker compose ps
```

Expected services:

- Frontend: `http://localhost:3000/login` displays the SmartFINN login page.
- Backend: `http://localhost:8000/api/v1/health` returns an HTTP 200 JSON health response.

Verify both endpoints with `curl --fail http://localhost:8000/api/v1/health` and `curl --fail --head http://localhost:3000/login`. If either fails, inspect `docker compose ps` and `docker compose logs --tail=200 backend frontend`.

8. Functional smoke test

- 1 Sign in with the active test staff account.
- 2 Confirm the dashboard and Applications page load.
- 3 Create an application with a synthetic applicant name.
- 4 Upload only files from `mock-data/`; never upload real personal information.
- 5 Confirm each uploaded document progresses to an extracted or clearly reported failed state.

- 6 Review extracted values and save one correction.
- 7 Approve the submission after all required documents are ready.
- 8 Start the risk assessment and open its traceable per-document results.
- 9 Confirm the displayed values mask bank account numbers where applicable.

The detailed expected results are in docs/ACCEPTANCE_TEST_REPORT.md.

9. Automated verification

Backend tests:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python -m pytest -q --cov=app --cov-report=term-missing --cov-fail-under=75
cd ..
```

On Windows PowerShell, activate with `.\.venv\Scripts\Activate.ps1`.

Frontend checks:

```
cd frontend
npm ci
npm run test:coverage
npm run lint
npm run build
cd ..
```

10. Stop and clean up

Stop the containers without deleting cloud data:

```
docker compose down
```

To remove only the locally built images as well:

```
docker compose down --rmi local
```

Remove synthetic test applications through the approved test-environment process. Do not delete shared Supabase data unless the project owner has authorised it.

11. Troubleshooting

Symptom	Likely cause	Action
Login succeeds in Supabase but SmartFINN returns 403	Missing or inactive <code>staff_profile</code>	Run the administrator bootstrap for the test user and verify <code>is_active</code>
Browser cannot call the API	Incorrect origin or port	Use the supplied local Compose file and open <code>http://localhost:3000</code>
OCR returns configuration error	Missing Azure values	Complete the Azure variables and restart the backend
OCR returns 429 or times out	Azure quota or service delay	Wait, retry once, and record the external failure if it persists
Upload is rejected	Unsupported content or size	Use the synthetic fixtures and keep files below 20 MB
Risk assessment returns conflict	Submission is not approved or required extraction is incomplete	Complete review and approval first
Docker service is unhealthy	Configuration or startup failure	Inspect <code>docker compose logs --tail=200 backend frontend</code>

12. Production deployment

Production uses `docker-compose.prod.yml`, host Nginx and the manual deployment workflow. Follow `docs/VPS_DEPLOYMENT.md`; do not use production credentials for marking unless the tutor has explicitly approved that arrangement.